

Task 1 — Trace the Code

x=1

y=2

1)x=x+y -> x=3

2)y=x*2 -> y= 6

3)x=y-x -> x=3

x=3 y=6

Task 2 — Find the Bug

Bug is : total+=numbers[i]

The code is used for finding the sum of numbers

Task 3 — Nested Loops (5 min)

a) $10 \times 10 = 100$ times

b) $1000 \times 1000 = 1_000_000$ times

c) $10 \times 1,000,000 = 10,000,000$ times

Task 4 — Which Is Faster? (5 min)

Algorithm A is faster , because time complexity is n , while Algorithm B takes $n \times n$ times complexity

Task 5 — Complexity Ranking (7 min)

1 -log n - n - n log n - n^2 - 2^n

Task 6 — Searching (7 min)

- 1) if array is not sorted I have to check all values , from 0 to n
- 2) if array is sorted I can use middle value to found it , $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$.

Task 7 — Large Dataset (8 min)

Algorithm: Scan all numbers once and keep only the current maximum.

Storage: No need to store all 500 million numbers.

Task 8

I will use HashMaps<String , Integer>

Task 9

$A+B = [[6,8][10,12]]$

$A \times B = [[19,22]43,50[]]$

Task 10

Memory access is slow — the array is huge and may not fit in CPU cache.

Memory bandwidth is limited — CPU waits for data to be loaded from RAM.

Cache misses — data is not in the CPU cache, causing delays.

Task 11

- The operating system finds the program file.
- It loads the program into **RAM**.
- The OS creates a **process** for the program.
- The CPU starts executing the instructions.
- The program may use memory, files, or other resources.
- The CPU processes the instructions and produces the result.
- The output is sent to the screen.

Task 12

A: CPU bottleneck.

- **B:** Memory/RAM bottleneck.
- **C:** Disk/I/O bottleneck.
- **D:** Waiting for user input (I/O bottleneck).

Yes, the OS can run them concurrently.

What determines CPU time? The OS scheduler decides which process gets CPU time.

With one CPU core: Only one program executes at a time, but the OS rapidly switches between them, creating the appearance of simultaneous execution.

Task 13

No I dont , The program may be limited by memory, disk I/O, network, or inefficient algorithms.

Task 14

Use a more efficient algorithm/data structure.

Reduce unnecessary calculations.

Avoid repeated work; use caching.

Process data in parallel.

Improve database queries.

Task 15

A, B, D and E together take **9s**. Making them 10× faster saves:

$$9 - 0.9 = 8.1s$$

New total:

$$50 + 0.9 = 50.9s$$

Speedup:

$$59 / 50.9 \approx 1.16\times$$

So the program is only about **16% faster**.

Optimization should focus on C, because it takes **50 of the 59 seconds**.

Task 16

Algorithm:

- Scan all 10 million marks once.
- Keep **min**, **max**, **sum**, count ≥ 90 , and count < 40 .
- Use a **frequency array of size 101** to calculate the median.
- Find the median from the frequency counts.

Data structures:

- Variables for min, max, sum and counters.
- **frequency[101]** for marks 0–100.

Number of examinations:

- Main data: **1 pass = 10 million examinations.**
- Median: scan only **101 frequency values.**

Memory:

- Frequency array needs only **101 integers.**
- No need to store all 10 million marks.

Likely bottleneck: Reading the **10 million marks from memory/disk**, rather than the calculations.